Part IV
Advanced probability modelling

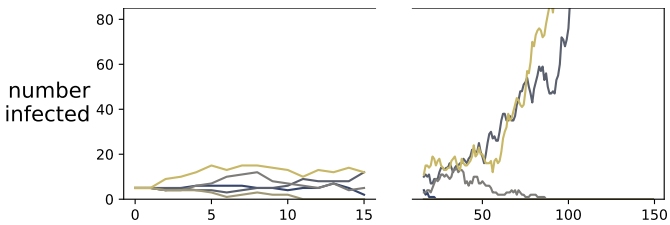
11. Markov chains

Science is often concerned with the laws that describe how a system changes over time. When we use probabilistic laws to describe these changes, the system is called a 'random process'. One very useful type of random process is a Markov chain, named after the Russian mathematician Andrei Markov (1856–1922) who invented them. The defining characteristic of a Markov chain is that the state of the system at time $n+1$ is a random variable generated based on the state at time n,

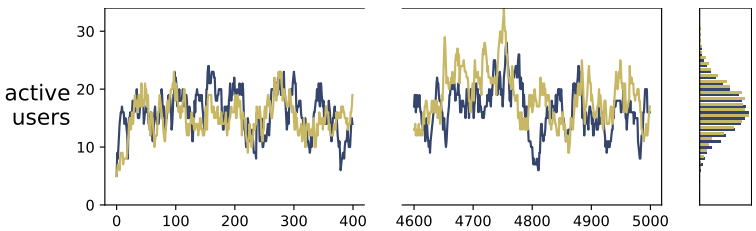
$$X_0 \to X_1 \to X_2 \to \cdots$$

For example, suppose we want to model the spread of an epidemic, and we let X_n be the number of infected people on day n. Then we can model X_{n+1} as a random variable, equal to X_n plus a random number of new infections, minus a random number of recoveries.

In this section we'll study tools for analysing Markov chain behaviour. To illustrate the types of behaviour we might see, the plot below shows five simulation runs of an epidemic model. In some of the runs the epidemic dies out, and in others it moves into exponential growth. Questions we might ask: How likely is it that an epidemic dies out? If it doesn't die out, what's the exponential growth rate? See sections 11.3 and 11.6 for the answers.



Other Markov chain models show completely different behaviour. The plot below shows two simulation runs of a model for the number of users currently using an online system: let X_n be the number of users at timestep n, and let X_{n+1} be X_n plus a random number of arrivals, minus a random number of departures. (The distributions we'd use here will be different to the distributions we'd use for the epidemic model, hence the different behaviour.)



On the left are the two runs, each shown over two timespans. On the right is a histogram, showing the long-run fraction of time spent in each state up to time 5000, for both of the runs. The number of active users hovers around 17, with fluctuations above and below. The two simulation runs have different fluctuations, but in aggregate they visit the same states with very nearly the same frequencies. Questions we might ask: What's the average number of active users? More fundamentally, how can we work out from a Markov chain model description whether it'll behave like epidemic model or like the active-user model? See sections 11.4 and 11.5 for the answers.

* ✻ *

The big modelling advance with Markov chains, compared to the models we've studied through the rest of the course, is that here we have a collection of random variables $(X_0, X_1, X_2, \dots)$ that are *not independent*. It is because of non-independence that random processes can show such a wide range of behaviours. It's what makes them useful models for describing systems, from epidemics to computer systems to financial markets to climate. In section 11.1 we'll survey some of these applications.

In the rest of your studies, you will come across Markov chains and other random process models in several courses:

- in *Computer Systems Modelling* they are used to describe discrete event simulations of communications networks
- in *Machine Learning and Bayesian Inference* they are used for numerically computing posterior distributions
- in *Probabilistic Machine Learning* they are extended to become recurrent neural networks, useful for language modelling
- in *Information Theory* they are used to describe noisy communications channels, and also the data streams sent over such channels

In IA *Discrete Mathematics* you learnt about finite automata. What is the relationship to Markov chains?

- Finite automata and Markov chains both have a set of possible states, and a lookup table / transition relation that describes progression from one state to the next.
- Finite automata are for describing algorithms that accept input, so the lookup table specifies 'what happens next, based on the current state and the given input symbol?' Markov chains are for describing systems that evolve by themselves, without input.
- Non-deterministic finite automata allow there to be several transitions out of a state, but they do not specify the probability of each transition, since they are intended to model 'what are all the things that might happen?' Markov chains do specify the transition probabilities, since they are intended to model 'what are the things that typically happen?'
- Markov chains are allowed to have an infinite state space, e.g. the space of all integers, or of real numbers.

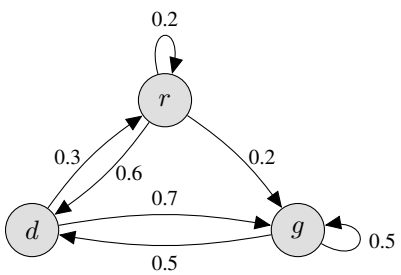
11.1. Applications of Markov chains

Markov chains are useful because they're simple enough to allow a good range of analyses yet rich enough to capture interesting behaviours. They have been used to model epidemics, climate, natural language, stock markets, and a vast range of other systems. In this section we'll look at some examples. (There are also plenty of interesting random processes that are *not* Markov chains, and we'll see an example in section 12.)

11.1.1. RANDOM WALK ON A GRAPH

The canonical Markov chain is a random walk on a directed graph. Imagine a particle that jumps from vertex to vertex, at each timestep selecting an outgoing edge randomly. We can let each edge have a weight, denoting the probability of choosing that edge. (Obviously the total weight of all edges out of a vertex must be 1.) The vertices are also called *states*, since they are the possible states that the particle can be in.

Example 11.1.1 (Cambridge weather).
The winter weather in Cambridge varies from grey (g) to drizzle (d) to rain (r). Suppose that the weather changes from day to day according to a Markov chain. We can specify the jump probabilities either by drawing a labelled graph (called the *state space diagram*), or by giving a matrix P (called the *transition matrix*) where P_{ij} is the probability that the particle, when in state i, picks state j to jump to next.



$$P = \begin{array}{c} \\ r \\ d \\ g \end{array} \begin{array}{c} \begin{array}{ccc} r & d & g \end{array} \\ \begin{pmatrix} 0.2 & 0.6 & 0.2 \\ 0.3 & 0 & 0.7 \\ 0 & 0.5 & 0.5 \end{pmatrix} \end{array}$$

Here's code to generate a sequence from this Markov chain. It also runs a sanity check, to ensure that every row of the transition matrix sums to 1.

```
# Let rain=0, drizzle=1, grey=2
P = np.array([[.2,.6,.2], [.3,0,.7], [0,.5,.5]])
assert np.allclose(np.sum(P, axis=1), np.ones(len(P))) # sanity check

def rweather(initial_state):
    x = initial_state
    while True:
        yield x
        x = np.random.choice(3, p=P[x])

x = rweather(initial_state=2)  # x is an infinite-lazy-list object
for n in range(100):
    print("On day", n, "weather is", next(x))
```

□

Random walk on the web. A famous application of random walks on graphs is to the world wide web. Let each vertex represent a web page, and let edges represent hyperlinks to other pages. Consider a random web surfer who browses randomly, from each page clicking on a link uniformly at random (or, if the current page has no outgoing links, restarting at a randomly chosen web page). This random web surfer will visit some pages more often than others, based on how well-linked each page is. This metric, 'how much time a random surfer spends at a given page', is the basis for Google's original PageRank system.

The same idea can be applied to other linked collections of documents. For example, let each vertex represent an article of a country's legal code, and let edges represent cross references to other articles. PageRank on this random walk will to measure how 'central' each article is.

11.1.2. SYSTEM MODELS

Example 11.1.2 (Simple epidemic model).
Let X_n be the number of infected people on day n. Each day some of the currently infected people recover, and some other people might become infected. Consider the model

$$X_{n+1} = X_n - \text{Bin}(X_n, p) + \text{Pois}(\lambda X_n).$$

The $\text{Bin}(X_n, p)$ term represents recoveries. It says that, on a given day, each infected person recovers with probability p. From the perspective of a single infected person, each day they have probability p of recovering. This implies that the time until they recover is a Geometric random variable with parameter p; this has mean $1/p$.

The $\text{Pois}(\lambda X_n)$ term represents new infections. The Poisson random variable[63] is a discrete random variable (as we'd hope, since we're using it to count numbers of people!) and it has expected value λX_n. So the expected number of new infections *per person per day* is λ. The Poisson random variable has a handy property: the sum of independent Poisson random variables is also Poisson, $\text{Pois}(x) + \text{Pois}(y) \sim \text{Pois}(x + y)$. So the new infections term can be thought of as the sum of X_n independent $\text{Pois}(\lambda)$ random variables. In other words, each infectious person infects $\text{Pois}(\lambda)$ different people per day.

Since each infected person infects on average λ people per day, and the average duration of infection is $1/p$, the expected number of new infections per person *over their entire illness* is $\lambda \times 1/p$. This is what's called the R number. Let's rewrite the model as

$$X_{n+1} = X_n - \text{Bin}(X_n, 1/d) + \text{Pois}(rX_n/d).$$

This is a more natural way to write it, since it's written using parameters that people can grasp more intuitively—the average duration of an infection $d = 1/p$, and the R number $r = \lambda/p$.

This epidemic model can be enhanced, for example by keeping track of three counts: S_n the number of people currently susceptible to infection, I_n the number currently infected, and R_n the number of people who are currently immune, e.g. because they had been infected and have recovered. This is called the SIR epidemic model. It's more useful than the simple epidemic model when there is a limited pool of people susceptible to infection, e.g. because of high vaccination rates or herd immunity.

Example 11.1.3 (Active-users model).
Here's a model for the number of users currently using an online platform: let X_n be the number of users at timestep n, and model it as

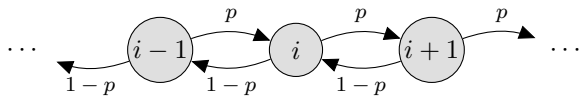
$$X_{n+1} = X_n + \text{Pois}(\lambda) - \text{Bin}(X_n, 1/d).$$

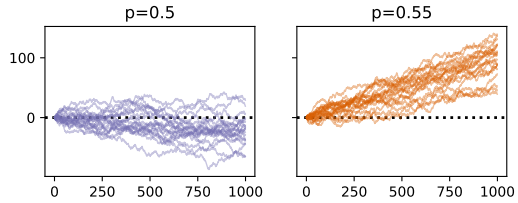
We model the number of users who depart the system each timestep as $\text{Bin}(X_n, 1/d)$, similar to recoveries in the epidemic model. We model new arrivals in a timestep as $\text{Pois}(\lambda)$. This means that the average number of new users in each timestep is constant (in contrast to the pidemic model, where the number of new infections was proportional to X_n).

There is a continuous-time version of this model, introduced by the Danish mathematician Agner Erlang (1878–1929) who invented the field of queueing theory; he used this model to analyse the number of active calls at a telephone exchange. Another variant, introduced by Allan Scherr in his PhD thesis at MIT, can be used to model time-sharing operating systems; with this work he invented the field of computer system performance evaluation.

11.1.3. SIMPLE RANDOM WALKS

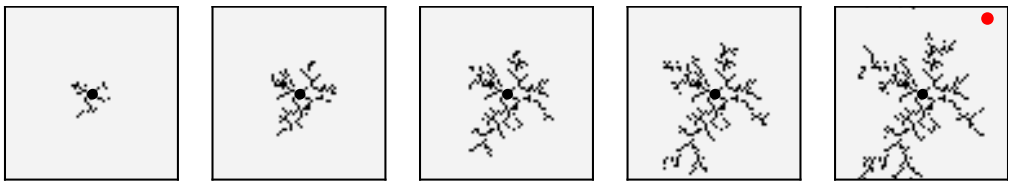
Example 11.1.4 (Simple random walk on $\mathbb{Z}$).
This is traditionally described as a drunkard attempting to walk home. They start at some initial location, call it 0, and then each timestep they stagger either left or right, right with probability p and left with probability $1-p$. This is called a *simple random walk on* $\mathbb{Z}$. The plots show 20 simulations each for two different values of p.





Let X_n be the location at timestep n, and let $d > 0$ be the location of the drunkard's house. Interesting question to ask: Let $D_n = \max_{0 \le u \le n} X_u$. How big does n need to be for $\mathbb{P}(D_n \ge d)$ to be larger than some threshold, say 95%? In other words, how long does the drunkard take to get home?

Example 11.1.5 (Simple random walk on $\mathbb{Z}^2$).
Here's a Snowflake Simulator. We start with a snowflake consisting of a single seed at the center of the image, and a particle placed randomly on the boundary. Each timestep the particle takes a step at random, up, down, left or right, except that it's not allowed outside the boundary. It moves randomly until it touches the snowflake, at which point it attaches. Start a new particle on the boundary, and repeat.



A typical question: rather than simulating the particle step-by-step, couldn't we just run a single computation to calculate the chance of every attachment point? This should speed up the simulation.

The snow particle performs a random walk on $\mathbb{Z}^2$, a Markov chain. But this example also involves a second higher-level Markov chain. Let S_n be the snowflake after n particles have attached themselves; this is an object in $\{0,1\}^{B \times B}$ where B is the size of the box. The snowflake attachment mechanism means that S_{n+1} is generated randomly with a distribution that depends only on S_n, in other words, $S_0 \to S_1 \to \cdots$ is a Markov chain on the state space of all possible snowflakes. Very challenging question: how does the diameter of S_n grow with n?

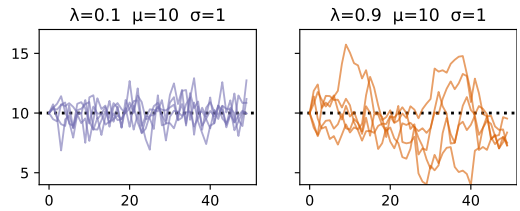
11.1.4. RED NOISE

Markov chains can have continuous state spaces, not just the discrete state spaces we have seen in the examples so far. Here is a simple example:

Example 11.1.6 (Mean-reverting random walk).
Consider the random process

$$X_{n+1} = \mu + \lambda(X_n - \mu) + N(0, \sigma^2) \quad \text{where} \quad 0 < \lambda < 1.$$

Our convention is that random variables are taken to be independent (unless specified otherwise), so the notation here means that the $N(0, \sigma^2)$ noise is independent every timestep.



This random process 'reverts' towards a value μ: if $\lambda \approx 0$ then it reverts almost immediately, and if $\lambda \approx 1$ then it reverts slowly and so excursions away from μ are longer-lived.

The mean-reverting random walk can be used to model long-lived fluctations. For example, we have previously considered the climate model

$$Y_t \sim \alpha \sin\big(2\pi(t + \phi)\big) + c + \gamma t + \text{Normal}(0, \sigma^2)$$

which says that each temperature reading Y_t is modelled by a deterministic term plus independent noise each timestep, also called 'white noise'. This would not be a good model for day-to-day temperatures, since hot spells and cold spells tend to last for several days, which means that the noise is not independent from day to day. We could instead model it as

$$Y_t = \alpha \sin\big(2\pi(t + \phi)\big) + c + \gamma t + E_t$$

where E_t is a mean-reverting random walk with mean 0, also called 'red noise'.

11.2. Calculations with Markov chains

When we're doing calculations with Markov chains, there are two things we know. We know that each item in the sequence X_{n+1} is generated based *only* on the preceding item X_n—that's the defining feature of Markov chains, and it's known as *memorylessness*. And we know the transition probabilities, because that's how we specify a Markov model. The general strategy is to wrangle the expression we want to calculate into a form where we can apply these two pieces of knowledge.

Memorylessness. To illustrate memorylessness, suppose we want to work out the probability $\mathbb{P}(X_2 = x_2 \mid X_1 = x_1, X_0 = x_0)$. Since X_2 is generated based only on X_1, the extra piece of information that $X_0 = x_0$ is irrelevant, and so

$$\mathbb{P}(X_2 = x_2 \mid X_1 = x_1, X_0 = x_0) = \mathbb{P}(X_2 = x_2 \mid X_1 = x_1).$$

A more formal way to state memorylessness[64] is that

$$\mathbb{P}(X_n = x_n \mid X_{n-1} = x_{n-1}, \ldots, X_0 = x_0) = \mathbb{P}(X_n = x_n \mid X_{n-1} = x_{n-1})$$

for any $(x_0, x_1, \ldots, x_n)$. In words, "conditional on the present, the future is independent of the past".

Conditioning. To leverage memorylessness, if the 'conditional on the present' term isn't there in the expression we want to calculate, we can introduce it via the law of total probability (LoTP). For example, if we want to calculate $\mathbb{P}(X_2 = x_2 \mid X_0 = x_0)$, we would introduce X_1 by

$$\mathbb{P}(X_2 = x_2 \mid X_0 = x_0) = \sum_{x_1} \mathbb{P}(X_2 = x_2 \mid X_1 = x_1, X_0 = x_0)\ \mathbb{P}(X_1 = x_1 \mid X_0 = x_0).$$

There are actually two forms of the law of total probability: the elementary version

$$\mathbb{P}(A = a) = \sum_b \mathbb{P}(A = a \mid B = b)\ \mathbb{P}(B = b)$$

and the 'with baggage' version:

$$\mathbb{P}(A = a \mid C = c) = \sum_b \mathbb{P}(A = a \mid B = b, C = c)\ \mathbb{P}(B = b \mid C = c)$$

The second version has 'baggage' $(\ \mid C = c)$ carried along in every term. All the fundamental laws of probability have a 'with baggage' version, and they're summarised on page 191. In the equation above, we're carrying along the baggage $(\ \mid X_0 = x_0)$.

Transition matrix. The point of these tools is that they let us rewrite our expression of interest into a form that's based on transition probabilities. We typically specify a Markov chain model by giving the transition matrix (P_{ij} is the probability that, from state i, it next jumps to state j) and once we've rewritten our expression of interest as a computation on the P matrix then that's as far as we can go with algebra. So, for example,

$$\begin{aligned}
&\mathbb{P}(X_1 = x_1 \mid X_0 = x_0) = P_{x_0\, x_1} \\
&\mathbb{P}(X_2 = x_2 \mid X_1 = x_1, X_0 = x_0) = \mathbb{P}(X_2 = x_2 \mid X_1 = x_1) = P_{x_1\, x_2}.
\end{aligned}$$

Resetting the clock. In all the Markov chain models we've looked at, the transition probabilities are the same at every timestep. Such a Markov chain is called *time-homogeneous*. This means that we can 'reset the clock' when we're doing probability calculations. For example, $\mathbb{P}(X_{103} = x \mid X_{100} = y)$ is equal to $\mathbb{P}(X_3 = x \mid X_0 = y)$. We implicitly used this above, when we said that that $X_1 \to X_2$ uses the same transition matrix as $X_0 \to X_1$. We'll see a more sophisticated use in section 11.3.

Time homogeneity isn't a required feature. We could for example invent a Markov chain that used a different transition matrix every timestep. As long as our model satisfies memorylessness, it's still called a Markov chain.

[64] The equations here use probabilities ($\mathbb{P}$) rather than likelihoods (Pr), so they only make sense for Markov chains with a discrete state space. The same ideas apply to Markov chains with a continuous state space, such as the red noise model in example 11.1.6, and the likelihood notation covers both cases. We will however in many of our calculations in sections 11.3–11.5 restrict attention to Markov chains with a discrete state space. When the calculations refer to a *transition matrix*, or when they state results about probabilites ($\mathbb{P}$), it is implied that we're making this restriction. This is to make it easier to wrap our heads around the notation—it's just more immediate to grasp $\mathbb{P}(X_2 = x_2 \mid X_1 = x_1, X_0 = x_0)$ than $\mathrm{Pr}_{X_2}(x_2 \mid x_1, x_0)$. It also reduces the mathematical overhead of technicalities arising from uncountable state spaces.

Example 11.2.1 (Multi-step transition probabilities).
Consider the Markov model for Cambridge weather on page 185, which has state space $\{r, d, g\}$ and transition matrix

$$P = \begin{array}{c} \\ r \\ d \\ g \end{array} \begin{array}{c} \begin{array}{ccc} r & d & g \end{array} \\ \begin{pmatrix} 0.2 & 0.6 & 0.2 \\ 0.3 & 0 & 0.7 \\ 0 & 0.5 & 0.5 \end{pmatrix} \end{array}$$

If it's grey (g) today, what's the chance of rain (r) two days from today? In other words, if $X_0, X_1, \ldots$ is a Markov chain with this transition matrix, what is $\mathbb{P}(X_2 = r \mid X_0 = g)$?

$$\begin{aligned}
\mathbb{P}(X_2 = r \mid X_0 = g) &= \sum_x \mathbb{P}(X_2 = r \mid X_1 = x, X_0 = g)\, \mathbb{P}(X_1 = x \mid X_0 = g) \\
&\qquad \textit{by LoTP with baggage } X_0 = g \\
&= \sum_x \mathbb{P}(X_2 = r \mid X_1 = x)\, \mathbb{P}(X_1 = x \mid X_0 = g) \\
&\qquad \textit{by memorylessness} \\
&= \sum_x P_{xr} P_{gx} \quad \textit{by definition of transition matrix} \\
&= \left[P^2\right]_{gr} \quad \textit{rewriting the sum in matrix notation.}
\end{aligned}$$

The result generalizes to n-step transition probabilities: $\mathbb{P}(X_n = j \mid X_0 = i) = \left[P^n\right]_{ij}$. Here's Python code:

```
P = np.array([[0.2, 0.6, 0.2], [0.3, 0, 0.7], [0, 0.5, 0.5]])
# Compute P^2 then pick out element at [2,2]
(P @ P)[2,0]
# More general code, for any number of steps n
n = 2
np.linalg.matrix_power(P, n)[2,0]
# returns the answer: 0.15
```

□

PROBABILITY RULES 'WITH BAGGAGE'

All the basic rules of elementary probability also have a 'conditional form'—where we stick 'conditional on C' onto every term, for an event C with $\mathbb{P}(C) > 0$. Conditional probability:

$$\mathbb{P}(A|B) = \frac{\mathbb{P}(A,B)}{\mathbb{P}(B)} \text{ if } \mathbb{P}(B) > 0 \qquad \mathbb{P}(A|B,C) = \frac{\mathbb{P}(A,B \mid C)}{\mathbb{P}(B|C)} \text{ if } \mathbb{P}(B|C) > 0$$

Independence of A and B; and conditional independence of A and B given C:

$$\mathbb{P}(A,B) = \mathbb{P}(A)\ \mathbb{P}(B) \qquad \mathbb{P}(A,B \mid C) = \mathbb{P}(A|C)\ \mathbb{P}(B|C)$$
$$\mathbb{P}(A|B) = \mathbb{P}(A) \qquad \mathbb{P}(A|B,C) = \mathbb{P}(A|C)$$

Law of total probability, for events $\{B_1, B_2, \dots\}$ that partition the sample space:

$$\mathbb{P}(A) = \sum_i \mathbb{P}(A|B_i)\ \mathbb{P}(B_i) \qquad \mathbb{P}(A|C) = \sum_i \mathbb{P}(A \mid B_i, C)\ \mathbb{P}(B_i|C)$$

Bayes's rule:

$$\mathbb{P}(A|B) = \frac{\mathbb{P}(B|A)\ \mathbb{P}(A)}{\mathbb{P}(B)} \qquad \mathbb{P}(A \mid B, C) = \frac{\mathbb{P}(B \mid A, C)\ \mathbb{P}(A|C)}{\mathbb{P}(B|C)}$$

Don't think of the conditional versions as separate rules, just think of them as "the basic rules of probability, which still work even if there's baggage being carried around".

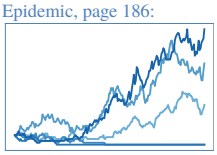
Each of these rules has a corresponding version for random variables. For example, the law of total probability turns into

$$\begin{aligned}
\mathrm{Pr}_X(x) &= \int_y \mathrm{Pr}_X(x \mid Y = y)\ \mathrm{Pr}_Y(y)\, dy \\
\mathrm{Pr}_X(x \mid C) &= \int_y \mathrm{Pr}_X(x \mid Y = y, C)\ \mathrm{Pr}_Y(y \mid C)\, dy \\
\mathrm{Pr}_X(x \mid Z = z) &= \int_y \mathrm{Pr}_X(x \mid Y = y, Z = z)\ \mathrm{Pr}_Y(y \mid Z = z)\, dy
\end{aligned}$$

Again, don't think of these as separate rules, just think of them as "the way to write the rules of basic probability so that they work with continuous random variables". The first version, for $\mathrm{Pr}_X(x)$, tells us something useful even when X or Y is continuous. The second version, for $\mathrm{Pr}_X(x \mid C)$, is another instance of "all the rules still work if we stick on the baggage 'conditional on C', as long as $\mathbb{P}(C) > 0$". The third version, for $\mathrm{Pr}_X(x \mid Z = z)$, is what we need if we want to condition on $C = \{Z = z\}$, an event that might have probability zero.

11.3. Hitting probabilities

Epidemic, page 186:



We might want to find the probability that a Markov chain eventually hits some state. For example, what is the probability that our epidemic model dies out?

There is a general theorem about hitting probabilities. You can find this and other results about hitting times etc. in standard textbooks on Markov chains[65]—though often it's just as much work to translate your problem into a theorem-ready version as it is to just solve it from first principles. Also, there will be many applications, especially in machine learning applied to Markov chains, where there aren't ready-made theorems. This theorem is included here for completeness, but it's the from-scratch calculation below that is more useful.

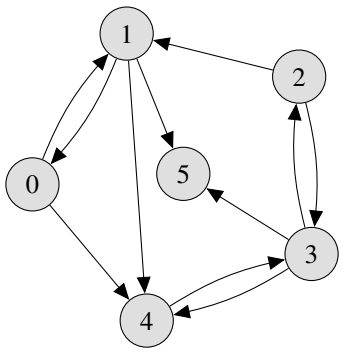
Theorem (hitting probability). *Consider a Markov chain on a discrete state space, with transition matrix P. Let A be a subset of a Markov chain's state space. The hitting probability from x is*

$$h_x = \mathbb{P}\big(\textit{ever hit } A \mid \textit{start at } x\big).$$

The hitting probabilities solve[66]

$$h_x = \sum_y P_{xy} h_y \ \textit{for all } x \notin A\,, \quad h_x = 1 \ \textit{for all } x \in A\,.$$

Example 11.3.1 (Hitting probabilities).
A web surfer goes from page to page, picking an outgoing link at random, until they stumble upon Twitter (page 5) which they never leave. Let h_x be the probability that, starting from x, they hit your website (page 0) before they get stuck on Twitter (so $h_0 = 1$ and $h_5 = 0$). Find h_x for all x.



We'll solve this problem from scratch, rather than by using the theorem.

To turn this question into a problem about Markov chains, we'll add an edge from node 5 back to itself. (Markov chains are infinite sequences: they're not allowed to just stop. By adding this edge we don't change the probabilities we're after, and we turn the web surfer into a Markov chain.) Let X_n be the page that the web surfer is on after n clicks, and write X for the entire process $X = (X_n)_{n\geq 0}$. Let P be the transition matrix. Then

$$\begin{aligned}
h_x &= \mathbb{P}(X \textit{ hits } 0 \mid X_0 = x) \quad \textit{by definition of } h \\
&= \sum_{x_1} \mathbb{P}(X \textit{ hits } 0 \mid X_1 = x_1, X_0 = x_0)\ \mathbb{P}(X_1 = x_1 \mid X_0 = x_0) \\
&\qquad \textit{by LoTP with baggage } \{X_0 = x\}, \textit{ for } x \neq 0, 5 \\
&= \sum_{x_1} \mathbb{P}(X \textit{ hits } 0 \mid X_1 = x_1)\, P_{xx_1} \quad \textit{by memorylessness, and substituting in } P \\
&= \sum_{x_1} \mathbb{P}(X \textit{ hits } 0 \mid X_0 = x_1)\, P_{xx_1} \quad \textit{by resetting the clock} \\
&= \sum_{x_1} h_{x_1} P_{xx_1} \quad \textit{by definition of } h.
\end{aligned}$$

[65] J. R. Norris. *Markov Chains*. Cambridge Series in Statistical and Probabilistic Mathematics. Cambridge University Press, 1997. URL: `http://www.statslab.cam.ac.uk/~james/Markov/`.

[66] Sometimes this system of equations has multiple solutions. In that case, the hitting probability from x is the minimum of h_x over all solutions $h \geq 0$.

In matrix notation,

$$\begin{bmatrix} h_0 \\ h_1 \\ h_2 \\ h_3 \\ h_4 \\ h_5 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ P_{10} & P_{11} & P_{12} & P_{13} & P_{14} & P_{15} \\ P_{20} & P_{21} & P_{22} & P_{23} & P_{24} & P_{25} \\ P_{30} & P_{31} & P_{32} & P_{33} & P_{34} & P_{35} \\ P_{40} & P_{41} & P_{42} & P_{43} & P_{44} & P_{45} \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} h_0 \\ h_1 \\ h_2 \\ h_3 \\ h_4 \\ h_5 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

The top row and the bottom row encode that $h_0 = 1$ and $h_5 = 0$, and the rest of the matrix implements $h_x = \sum_y P_{xy} h_y$ for $x \neq 0, 1$. We can solve this in Python:

```
# Let E be the adjacency matrix for the graph
E = np.array([[0,1,0,0,1,0],
              [1,0,0,0,1,1],
              [0,1,0,1,0,0],
              [0,0,1,0,1,1],
              [0,0,0,1,0,0],
              [0,0,0,0,0,1]])
# For a vertex with n outgoing edges, let P_ij = 1/n on each of them
P = E / E.sum(axis=1)[:,np.newaxis]
# Sanity check that this is a valid transition matrix
assert np.allclose(P.sum(axis=1), 1)

# Set up the equations to solve, with modified rows for states 0 and 5:
# h = Qh + b, or equivalently (I - Q)h = b.
Q = P.copy()
Q[0,:] = 0
Q[5,:] = 0
b = np.zeros(6)
b[0] = 1
h = np.linalg.solve(np.eye(6) - Q, b)
```

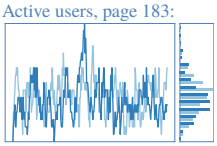
```
array([1. , 0.375, 0.25 , 0.125, 0.125, 0. ])
```

□

We could equally well have appealed to the general theorem. Let $A = \{0\}$, and then the theorem gives us exactly the same equations that we derived from scratch. The theorem is only really useful for Markov chains with infinite state spaces, when the hitting probability equations might have multiple solutions. In that case, the theorem tells us which solution is the right one.

11.4. Stationary distribution

Active users, page 183:



Out 'active users' model settles down to a stable distribution. How do we find that distribution? More importantly, what do we mean by 'stable distribution'?

Definition of stationarity. A distribution π over the state space of a Markov chain is called a *stationary distribution* or *equilibrium distribution* if

$$X_0 \sim \pi \qquad \Longrightarrow \qquad X_1 \sim \pi$$

resetting the clock: page 189

In other words, supposing that we choose the initial state X_0 randomly with distribution $\mathbb{P}(X_0 = x) = \pi_x$, then the next state X_1 will have the same distribution. Hence (by resetting the clock) $X_2 \sim \pi$, and so on, thus $X_n \sim \pi$ for all n. When $X_n \sim \pi$ for all n, we say the Markov chain itself is *stationary*.

The word 'stationary' does not mean that the Markov chain has somehow stopped—a Markov chain is defined to go on forever, always stepping randomly from state to state. It's not X_n that's stationary, it's the *distribution* $\Pr_{X_n}(\cdot)$ that is stationary i.e. unchanging.

Technically, all that stationarity tells us is 'if we pick the initial state X_0 randomly according to π, then X_1 will have distribution π, and X_2, and so on.' However, as we'll discuss in section 11.5, the fraction of time spent in any state x is given by the stationary distribution π_x. (In the plot at the top of this page, the histogram on the right is actually showing the time spent in each state.)

11.4.1. FINDING THE STATIONARY DISTRIBUTION

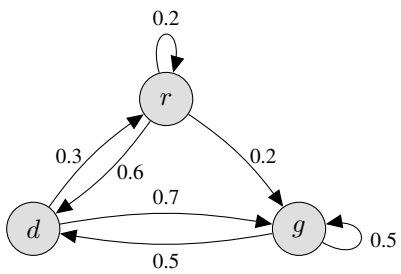
Stationarity equations. Consider a Markov chain with transition matrix P. If π is a stationary distribution, then it solves

$$\pi = \pi P, \quad \pi^\top 1 = 1.$$

Conversely, if π is a distribution that solves these equations then it is a stationary distribution.

Let's see how these stationarity equations can be derived, using nothing more that the definition of stationarity. We'll work through a concrete example.

Example 11.4.1 (Computing the stationary distribution).
Find the stationary distribution of Cambridge weather.



The definition of stationarity says that if we start time time 0 with distribution $X_0 \sim \pi$, then X_1 will have distribution π also. But there's another way to get the distribution of X_1, using the standard types of Markov chain calculations we saw in section 11.2:

$$\begin{aligned}\mathbb{P}(X_1 = x) &= \sum_y \mathbb{P}(X_1 = x \mid X_0 = y)\, \mathbb{P}(X_0 = y) \quad \textit{by Law of Total Prob.}\\ &= \sum_y P_{yx}\pi_y \quad \textit{since we assumed } X_0 \sim \pi\end{aligned}$$

thus

$$\pi_x = \sum_y P_{yx}\pi_y \quad \textit{since by stationarity } X_1 \sim \pi.$$

Writing out these equations, one for every state x,

$$\begin{aligned}\pi_r &= 0.2\pi_r + 0.3\pi_d \\ \pi_d &= 0.6\pi_r + 0.5\pi_g \\ \pi_g &= 0.2\pi_r + 0.7\pi_d + 0.5\pi_g.\end{aligned}$$

Although there are three equations and three unknowns, when we try to solve them we find there is not a unique solution: if the vector π is a solution then so is $\kappa\pi$ for any constant κ. There's nothing in these equations that says that π has to be a probability *vector. We can just add that in as an extra equation:*

$$\sum_x \pi_x = 1.$$

Rather than solve all these simultaneous equations with algebra, we can turn them into matrix notation and then ask the computer to solve them. The stationarity equations become $\pi = \pi P$, or equivalently $(P - I)^\top \pi = 0$. The normalizing equation is $1^\top \pi = 1$. In Python,

In numpy, if π is a one-dimensional array then it can be used either as a row vector or a column vector. In πP it is treated as a row vector, and in $(P - I)^\top \pi$ it is treated as a column vector.

```
P = np.array([[0.2, 0.6, 0.2], [0.3, 0, 0.7], [0, 0.5, 0.5]])
A = np.concatenate(((P - np.eye(3)).transpose(), [[1,1,1]]))
π = np.linalg.lstsq(A, [0,0,0, 1])[0]
```

This code specifies four *linear equations with only* three *unknowns. We could have removed one of the equations before asking numpy for a solution, but the lazier (and better!) approach is to just give it four equations and call* np.linalg.lstsq. *This library routine will find a least-squares solution for any system of equations: if the equations have multiple solutions then it will return one; if the equations have no solution then it'll return a least-squares compromise.*

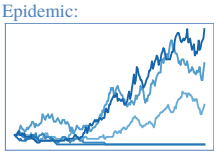
□

11.4.2. EXISTENCE AND UNIQUENESS.

We said *if* π is a stationary distribution *then* it solves the stationarity equations. We didn't say that the equations *can* be solved, nor that if they can then there's a unique solution. Here's a theorem that addresses these points.

Existence and uniqueness theorem. Suppose (1) the state space is finite, and (2) there is a path from any state to any other. (This second property is called *irreducibility*.) Then there exists a unique stationary distribution.

Epidemic:



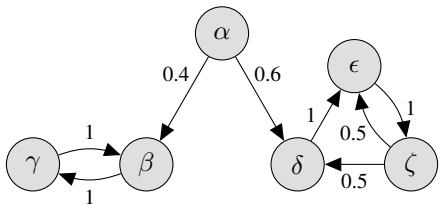
To see the intuition behind this, let's consider the epidemic model. This model doesn't impose any upper bound on the number of infected people, so the state space is infinite. A Markov chain with an infinite state space can *explode*, i.e. head off and never return, as three of these simulation runs look as if they're set to do. If the Markov chain can explode, there might not even exist a stationary distribution.

The other condition in the theorem is about irreducibility, the property that there's a path from any state to any other. The epidemic model isn't irreducible, since once it reaches the state where zero people are infected the infection is dead and the Markov chain is stuck at zero. (Such a state is called an *absorbing* state.) In Markov chains like this, the theorem doesn't apply; there might be no stationary distribution, or there might be mulitple solutions.

Active users:

The theorem says that *if* certain conditions are met *then* the chain has a stationary distribution. It's not an if-and-only-if. The active users model has an infinite state space (it doesn't impose any upper bound on the number of active users), but it doesn't explode, it just settled down, and it has a stationary distribution. It doesn't hurt to try and solve the stationarity equations even for an infinite-state-space Markov chain; there's no guarantee we can solve them, but if we can solve them then we must have a stationary distribution.

Exercise 11.4.2 (Multiple stationary distributions).
Show that this pathological Markov chain has multiple stationary distributions. Find them all.



We can compute a stationary distribution for the pathological Markov chain using exactly the same method, but there is a problem: the stationarity equations have multiple solutions, even after imposing the extra equation $\sum_x \pi_x = 1$. *If we just write out all the equations longhand,*

$$
\begin{aligned}
\pi_\alpha &= 0 \\
\pi_\beta &= 0.4\pi_\alpha + \pi_\gamma \\
\pi_\gamma &= \pi_\beta \\
\pi_\delta &= 0.6\pi_\alpha + 0.5\pi_\zeta \\
\pi_\epsilon &= \pi_\delta + 0.5\pi_\zeta \\
\pi_\zeta &= \pi_\epsilon \\
\pi_\alpha + \pi_\beta + \pi_\gamma + \pi_\delta + \pi_\epsilon + \pi_\zeta &= 1
\end{aligned}
$$

and solve these equations simultaneously, we discover that the general solution is

$$[\pi_\alpha, \pi_\beta, \pi_\gamma, \pi_\delta, \pi_\epsilon, \pi_\zeta] = a\left[0, {}^1\!/_2, {}^1\!/_2, 0, 0, 0\right] + (1-a)\left[0, 0, 0, {}^1\!/_5, {}^2\!/_5, {}^2\!/_5\right] \tag{2}$$

for any real value a *(though only* $a \in [0,1]$ *will yield a legitimate probability distribution). In Python, if we look carefully at the output of* np.linalg.lstsq() *and read the documentation, we see it telling us that the linear equation does not have a unique solution; there are further* np.linalg *tools that can extract the general form of the solution.*

Equation (2) *actually has a nice intuitive explanation. The Markov chain could be spending all its time in states* $\{\beta, \gamma\}$ *with stationary distribution* $[1/2, 1/2]$*, or it could be spending all its time in states* $\{\delta, \epsilon, \zeta\}$ *with stationary distribution* $[1/5, 2/5, 2/5]$.
□

Example 11.4.3 (Irreducibility).
Is the Markov chain from example 11.4.2 irreducible? What about the Cambridge weather model in example 11.4.1?

The pathological Markov chain is not irreducible, because it is impossible to get from β *to* α.

The Cambridge weather Markov chain can get from any state to any other state; to get from g *to* r *takes two steps, and all the others can be achieved in one step. Therefore it is irreducible, therefore it has a unique stationary distribution.*
□

11.4.3. DETAILED BALANCE

Often, when we want to find the stationary distribution, there's nothing for it but to use np.linalg and solve a matrix equation. In some special cases the Markov chain has a form that lets us find the stationary distribution with very little algebra, using a trick called *detailed balance*.

This seems like a curiosity, not worth mentioning in a data science course—except that there is a powerful tool for computational Bayesian calculation, much more powerful than the crude tool we've used in this course, which hinges on detailed balance. It's called Gibbs Sampling, and you'll learn about it next year in Part II *Machine Learning and Bayesian Inference*.

computational Bayes: section 6.2 page 100

Detailed balance equations. Let X be a Markov chain with transition matrix P. Suppose there is a distribution π such that the so-called 'detailed balance' condition is satisfied:

$$\pi_x P_{x\,y} = \pi_y P_{y\,x} \quad \text{for all states } x \text{ and } y$$

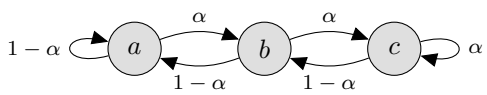
Then π solves $\pi = \pi P$, hence π is a stationary distribution.

It's easy to prove this result, using the fact that each row of the transition matrix must sum to 1:

$$[\pi P]_x = \sum_y \pi_y P_{yx} = \sum_y \pi_x P_{xy} = \pi_x \sum_y P_{xy} = \pi_x.$$

It doesn't hurt to try to solve the detailed balance equations; either we find the stationary distribution without much work, or we quickly discover that they can't be solved and we have to solve the full stationarity equations.

Exercise 11.4.4 (Stationary distribution using detailed balance).
Calculate the stationary distribution of the following Markov chain.



The detailed balance equations are

$$\begin{aligned}
&\textit{for } (a,b) \textit{ and } (b,a)\textit{:} \quad \pi_a \alpha = \pi_b (1-\alpha) \\
&\textit{for } (a,c) \textit{ and } (c,a)\textit{:} \quad \pi_a 0 = \pi_c 0 \\
&\textit{for } (b,c) \textit{ and } (c,b)\textit{:} \quad \pi_b \alpha = \pi_c (1-\alpha) \\
&\textit{for } (a,a) \textit{ etc.:} \quad \pi_a (1-\alpha) = \pi_a (1-\alpha) \textit{ etc.}
\end{aligned}$$

Some of these equations are redundant (the second equation just says $0 = 0$, the fourth is trivial). But there are enough equations that we can derive

$$\pi_b = \pi_a \frac{\alpha}{1-\alpha}, \quad \pi_c = \pi_a \Big(\frac{\alpha}{1-\alpha}\Big)^2.$$

(assuming $0 < \alpha < 1$). We want π to be a distribution, so we add the constraint $\pi_a + \pi_b + \pi_c = 1$, and get

$$\big[\pi_a, \pi_b, \pi_c\big] = \frac{1}{1 + \alpha/(1-\alpha) + \alpha^2/(1-\alpha)^2} \Big[1, \frac{\alpha}{1-\alpha}, \Big(\frac{\alpha}{1-\alpha}\Big)^2\Big].$$

The detailed balance theorem tells us that this is a *stationary distribution.*

Is it the *stationary distribution? in other words, is the stationary distribution unique? In the case $0 < \alpha < 1$, it's easy to see that it's possible to get from any state to any other. Therefore the existence and uniqueness theorem for stationary distributions applies, i.e. there is a unique stationary distribution, so it must be the one we found.*

In the cases $\alpha = 0$ or $\alpha = 1$ then the chain is not irreducible: if $\alpha = 0$ then it gets stuck in state a, so the stationary distribution is $\pi_a = 1$, $\pi_b = \pi_c = 0$; and if $\alpha = 1$ then it gets stuck in state c, so the stationary distribution is $\pi_a = \pi_b = 0$, $\pi_c = 1$. So, in both of these cases, there is still a unique stationary distribution.

□

Exercise 11.4.5 (Random walk on an undirected graph).

A knight moves on an otherwise empty chessboard, each timestep picking one of its legal moves at random (out of 8 legal moves if it is in the center of the board, and 2 legal moves if it is in a corner). Show that the stationary probability of being in position x is $m_x/336$, where m_x is the number of legal moves out of position x.

We should first check whether the Markov chain described in the question is irreducible, since otherwise there isn't even a unique stationary distribution. This is just a matter of sketching a chessboard and persuading ourselves that a knight can indeed get from any position to any other position, given enough moves.

The question tells us the stationary distribution and asks us to verify it. We could plug it into the full stationarity equations, but if it happens to solve the detailed balance equations then that is sufficient and our work will be simpler. The detailed balance equations are

$$\begin{aligned} \frac{m_x}{336} \times \frac{1}{m_x} &= \frac{m_y}{336} \times \frac{1}{m_y} \quad \text{if } x \leftrightarrow y \text{ is legal,} \\ \frac{m_x}{336} \times 0 &= \frac{m_y}{336} \times 0 \quad \text{if } x \leftrightarrow y \text{ is illegal.} \end{aligned}$$

These equations are certainly true, and they are the only equations that need to be satisfied, since $x \to y$ is legal if and only if $y \to x$ is legal. Therefore the suggested distribution solves detailed balance.

Finally, we need to verify that the suggested distribution is indeed a distribution, i.e. that it sums to 1. Counting the number of possible moves from every position on the chessboard gives a total of 336, thus $\sum_x m_x/336 = 1$.

□

The result described here can be generalized to a random walk on any undirected graph.

11.5. Stationarity and average behaviour

Active users:

In our Markov chain model for the number of active users of an online platform, a natural question is: when a new user turns up, what's the chance that the Markov chain is in state x? (Perhaps the system starts to run slowly when x is too large and we want to know how often this occurs.)

If the user's arrival time is random, any time equally likely, then this is a question about the time-average properties of the Markov chain. The probability of finding the system in state x is equal to the fraction of time spent in state x, averaged over a time window of duration n,

$$\frac{1}{n}\sum_{i=1}^{n} 1_{X_i=x}.$$

The plot in the margin shows traces from two simulation runs of our active users model, and for each trace it shows a histogram of the amount of time spent in each state. The two histograms are roughly the same, and in fact they both follow the stationary distribution.

Some simple algebra shows why there's a link between time-averages and stationary distributions. If π is a stationary distribution, and if the Markov chain starts in a random state $X_0 \sim \pi$, then the chain is stationary i.e. $X_i \sim \pi$ for all i, so $\mathbb{P}(X_i = x) = \pi_x$, and so

$$\mathbb{E}\Big(\frac{1}{n}\sum_{i=1}^{n} 1_{X_i=x}\Big) = \frac{1}{n}\sum_i \mathbb{E}\, 1_{X_i=x} = \frac{1}{n}\sum_i \mathbb{P}(X_i = x) = \pi_x \quad \text{for all } x \text{ and } n.$$

But this simple conclusion hides a nuance: that there are two different types of 'average' going on.

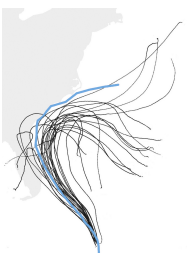
1. We could take a single trace, and measure the long-run fraction of time it spends in a particular state. This is a time average, sometimes called an *ergodic* average, and it's represented by the $n^{-1}\sum_{i=1}^{n}\{\cdot\}$ part of the expression.

   Different traces could conceivably have different long-run time averages; for example, maybe the system has a tipping point, and some simulation runs end up in one regime and some runs in another. Well-designed computer systems will hopefully not hve tipping points! Our active-users model, for example, doesn't have a tipping point; it can be shown that nearly every trace will have identical long-run time averages.

2. We could run many simulations, and measure the fraction of simulations that have some particular property. This is called an *ensemble* average, and it's represented by the $\mathbb{E}\{\cdot\}$ part of the expression, which is an average over all possible traces $(x_0, x_1, \ldots, x_n)$. It's also the type of average we mean when we write $\mathbb{P}(X_i = x)$.

   Climate science makes heavy use of ensemble averages. For example, we might run several simulations of a hurricane, and measure the mean and standard deviation of where the hurricane is at some particular point in time. Plotting the traces that make up an ensemble is a great visual tool for conveying uncertainty about a random process.

Actual trajectory of Hurricane Matthew (2016), plus simulated trajectories:[67]



For Markov chains with a stationary distribution (like the active-users model, and unlike hurricane simulators) we're generally interested in questions about the 'typical' state, rather than the state at some particular timepoint. In the remainder of this section we'll make precise mathematical statements about the typical state, one using time averaging and one using ensemble averaging, both of them tighter than our simple equation about expected time-averages.

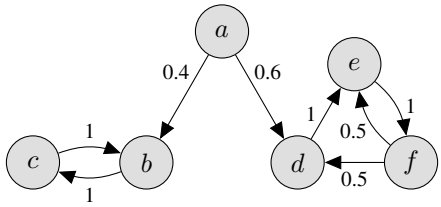
But the real message of this section is that Markov chains can have all sorts of strange behaviours, for example ensemble averages and time averages that disagree, and it needs a level head to even ask a well-formed question. Markov chains are a great starting point for thinking about random processes because they're rich enough to have these complex behaviours, and they're still simple enough to simulate and analyse.

[67] Plot taken from Jun Yang et al. "Incorporating inland flooding into hurricane evacuation decision support modelling". In: *Natural Hazards* (2019). DOI: 10.1007/s11069-019-03573-9.

FORMAL RESULTS

time-homogeneous: defined on page 189

To reduce the mathematical overhead we will restrict attention to time-homogeneous Markov chains with a finite state space. We'll illustrate with the pathological Markov chain below. It's useful to have a simple concrete example that pushes the limits, in order to help us sharpen our thinking.



Time averages for individual runs. In the active-users model, for *each* of the two individual simulations that we ran, we got roughly the same histogram for fraction-of-time in each state. There's a theorem behind this. Informally, with a well-behaved Markov chain, for pretty much *every* trace that we might generate, the histogram of time averages is approximately the stationary distribution π:

$$\frac{1}{n}\sum_{i=1}^{n} 1_{X_i=x} \to \pi_x \quad \text{as } n \to \infty.$$

irreducible: defined on page 196

Theorem.[68] *Let X be a Markov chain with stationary distribution π, and assume it is irreducible. Then for every state x, and regardless of the distribution of the inital state X_0,*

$$\mathbb{P}\Big(\lim_{n\to\infty} \frac{1}{n}\sum_{i=1}^{n} 1_{X_i=x} = \pi_x\Big) = 1.$$

What about badly-behaved Markov chains? In our pathological Markov chain, starting at state $X_0 = a$, the very first timestep is a tipping point. With probability 0.4 it'll jump to $X_1 = b$ and thereafter it'll divide its time equally between the two states on the left, and its time averages will be given by the stationary distribution $\pi_b^{(1)} = \pi_c^{(1)} = 1/2$. And with probability 0.6 it'll jump to $X_1 = d$ and thereafter it'll divide its time between the three states on the right, and its time averages will be given by a different stationary distribution, $\pi_d^{(2)} = 1/5$, $\pi_e^{(2)} = \pi_f^{(2)} = 2/5$. In maths notation,

$$\lim_{n\to\infty} \frac{1}{n}\sum_{i=1}^{n} 1_{X_i=x} = \begin{cases} \pi_x^{(1)} & \text{with probability } 0.4 \\ \pi_x^{(2)} & \text{with probability } 0.6. \end{cases}$$

What makes this Markov chain badly-behaved is that it's not irreducible, and it has multiple stationary distributions. Each individual run still does have time averages given by a stationary distribution, but some runs have one histogram of time averages, and other runs have a different histogram.

Ensemble averages at an arbitrary instant. We started this section with a question about the state at an instant in time: in our active-users model, at the instant a new user turns up, what's the chance they find the system in state x?

We've discussed one answer to this question, looking at time averages for a single trace. If we're interested in a new user who turns up at a random instant in time, time averaging is what matters. Now we'll look at another way to answer the question, looking at an *arbitrary* instant rather than a *random* instant, using ensemble averages.

Earlier, we observed that if the system starts in the stationary distribution $X_0 \sim \pi$, then for all n it has the same distribution, so $\mathbb{P}(X_n = x) = \pi_x$ at *every* instant. But the two simulations shown at the top of the previous page don't actually start at a random state, they both start at $X_0 = 0$ users. From the plots, it looks as if the simulations quickly 'forget' their initialization, so we might expect $\mathbb{P}(X_n = x \mid X_0 = 0) \approx \pi_x$ for all sufficiently large n. When this is true, then we might still say that π represents the state at a 'typical' instant.

(Such probability approximations are practically useful. In many applications, particularly in advanced Bayesian inference, we want to be able to generate a random variable from a distribution π that we can't even write out explicitly. Suppose we can cunningly devise transition probabilities for

[68] See theorems 1.7.7 and 1.10.2 in J. R. Norris. *Markov Chains*. Cambridge Series in Statistical and Probabilistic Mathematics. Cambridge University Press, 1997. URL: `http://www.statslab.cam.ac.uk/~james/Markov/`.

a Markov chain so that the stationary distribution is π. We could run an ensemble of Markov chains, choose some large n, and plot a histogram of X_n by aggregating over the ensemble, and this should be close to π. Alternatively, we could run a single Markov chain and read off X_n at time n, and this should be nearly the same as sampling from π. In such applications our goal is to be able to sample from π, so it would be defeating the purpose to say "*start* our simulation in random state $X_0 \sim \pi$"!)

Our pathological Markov chain shows that we need to be cautious. Suppose we start it in state $X_0 = b$. Thereafter $X_n = b$ for every even n, and $X_n = c$ for every odd n, and so there is no 'sufficiently large n' to forget the initial state, and there's no such thing as a 'typical' state. It's handy to be able to rule out pathological cases like this. Here's a definition:

A state x is said to be *aperiodic* if there exists an n_0 such that $\mathbb{P}(X_n = x \mid X_0 = x) > 0$ for all $n \geq n_0$. It's easy to show that if the chain is irreducible and has an aperiodic state, then all its states are aperiodic.

Theorem. *Let X be a Markov chain. If the chain is irreducible and has an aperiodic state, then*

$$\lim_{n\to\infty} \mathbb{P}(X_n = x \mid X_0 = x_0) = \pi_x \quad \text{for all states } x \text{ and } x_0.$$

Exercise 11.5.1. Consider the three-state Markov chain consisting of states d, e, and f from the pathological Markov chain. Show that all three states are aperiodic.

First we verify that it is irreducible. We can get from any state to any other by following links $d \to e \to f \to d \to \ldots$, so yes it is irreducible.

For aperiodicity, let's pick one state arbitrarily, say d, and work out if that state is aperiodic. The theorem says that if one state is aperiodic then all states are aperiodic. Can we get from d to d in n steps?

n	*can get from d to d in n steps?*
1	*no*
2	*no*
3	*yes, $d \to e \to f \to d$*
4	*no*
5	*yes, by adding the loop $e \to f \to e$ in the middle*
6	*yes, going around the loop $d \to e \to f \to d$ twice*
7	*yes, using the loop $e \to f \to e$ twice*
8	*yes, since $8 = 5 + 3$ and 5 and 3 are possible*
9	*yes, 3+3+3*
$n \geq 8$	*yes, by mixing loops of length 5 and length 3*

(This is related to IA Discrete Mathematics. *We can go from d to d in 3 steps, and in 5 steps. The greatest common divisor of 3 and 5 is 1, therefore there is an integer linear combination equal to 1, in this case $2 \times 3 - 1 \times 5 = 1$. We can achieve any $n = 5m$ by m copies of the 5-step loop, and we can achieve $n = 5m + l$ by adding l copies of the $2 \times 3 - 1 \times 5$ widget.)*

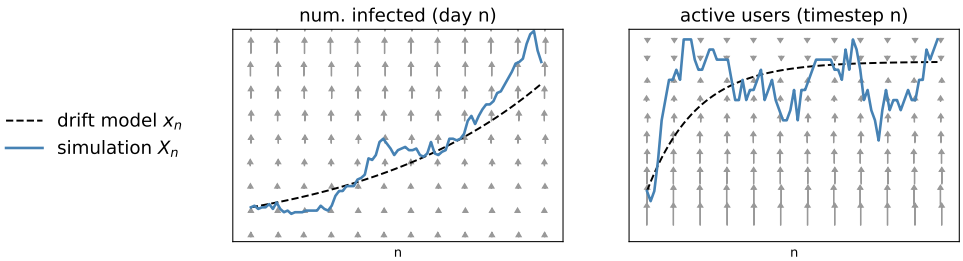
□

11.6. Drift analysis

Drift analysis is a nice simple back-of-the-envelope way to get a rough idea of how a Markov chain is likely to behave.

- The *drift* is the expected rate of change in a quantity: $\delta(x) = \mathbb{E}(X_{n+1} - X_n \mid X_n = x)$.
- A *drift diagram* uses arrows to show the direction and strength of the drift.
- A *drift model* is a deterministic approximation, from following the drift: $x_{n+1} = x_n + \delta(x_n)$.
- A *fixed point* is a state $\hat{x}$ where the drift is zero.

Here are drift diagrams for the two Markov chains from the beginning of this section,



epidemic model and active-users model, page 186

the epidemic model

$$\text{Markov chain:} \qquad X_{n+1} = X_n + \text{Pois}(rX_n/d) - \text{Bin}(X_n, 1/d)$$

$$\text{Drift:} \qquad \delta(x) = x\Big(\frac{r-1}{d}\Big)$$

and the active-users model

$$\text{Markov chain:} \qquad X_{n+1} = X_n + \text{Pois}(\lambda) - \text{Bin}(X_n, 1/d)$$

$$\text{Drift:} \qquad \delta(x) = \lambda - \frac{x}{d}$$

These drift diagrams use arrows to indicate the drift, an up-pointing arrow if $\delta(x) > 0$ and down-pointing if $\delta(x) < 0$. Interpret an up-pointing arrow as "if we're in a state where the drift is large positive, then the next state is likely to be a big increase on the current state". They give us a rough idea of how the Markov chain is likely to behave:

- For the epidemic model with $r > 1$, the drift is positive for any $x > 0$, so the Markov chain always tends to increase. And the drift model is easy to solve exactly: $x_n = x_0\big(1+(r-1)/d\big)^n$, in other words exponential growth with growth rate $(r-1)/d$ per day. For the parameters on page 183, the growth rate is 2% per day. The Markov chain will have fluctuations of course, since the Markov chain is random,[69] but it will have roughly the same growth rate as the drift model.
- For the active-users model, there is a fixed point at $\hat{x} = \lambda d$; for $x < \hat{x}$ the drift is positive, and for $x > \hat{x}$ the drift is negative. So the Markov chain has a tendency to be 'pushed' back towards $\hat{x}$. There are always fluctuations of course, since the Markov chain is random, but they're fluctuations around $\hat{x}$. For the parameters on page 183, the fixed point is $\hat{x} = 16.8$, roughly at the center of the histogram shown there.

Real-valued readout functions were also used for Monte Carlo integration, page 97, and hypothesis testing, page 159

For Markov chains with more complicated state spaces, it's trickier to reason about the drift. If the state space is two dimensional, we might plot a 'phase space diagram' with arrows showing the drift $\delta(x, y)$ at every state (x, y). For Markov chains with non-numerical state spaces, there might be some useful real-valued readout function $h(X_n)$, and we could plot the drift in $h(X_n)$.

[69] The standard deviation of $X_{n+1} - X_n$ is easy to calculate directly, since it is the sum of independent Poisson and Binomial random variables, and we can simply look up their variances on Wikipedia. The standard deviation of $X_{n+1} - X_n$, conditional on $X_n = x$, is proportional to $\sqrt{x}$. So, when X_n is large, the random fluctuations in X_n will be very small compared to the drift, therefore the Markov chain will follow the drift model very closely.

12. Learning a random process

Some data presents itself as *sequences*, and we're interested in learning how the sequence evolves in time. A natural example is a time series of sensor readings; but this class of data also includes much more such as log records and even text. Here's an example, a dataset of baby names—i.e. a collection of sequences of characters.

aaron	abraham	aditya
abdullah	adam	adonis
abel	addison	adrian
abigail	adeline	...

How might we model such data? The challenge is that the sample space consists of sequences of varying length, whereas nearly all the probability models we've seen in Part I have simple scalar outputs.

More importantly, what do we want such a model for? One simple use is to be able to generate new sequences (e.g. come up with novel baby names; build a climate simulator that generates realistic weather traces). Here are some more modelling goals:

- *Simple scientific curiosity, to learn the laws of how the sequence evolves.* Once we have probabilistic sequence models, we can estimate parameters for these laws, or use inference procedures to test whether a proposed law is consistent with the data. Section 12.1 describes a simple way to do parameter estimation.
- *Sequence completion.* If we have a probability model for random sequences, and we're given the first few items in a sequence, then we can in principle find the conditional distribution of the rest of the sequence. For example, if we have a sequence model trained on Wikipedia pages (the dataset being a collection of pages, each page a sequence of characters) we could like Snow White's stepmother give it a prompt "The␣country␣with␣the␣reputation␣for␣best␣food␣is␣" and ask it to generate the rest of the sequence conditional on this prefix. This is how language models such as GPT3 and GitHub Copilot work. Section 12.2 describes recurrent neural networks, the precursor to these language models.
- *More general filling-in-the-gaps.* We can in principle find the conditional distribution of any missing part of a sequence, given the rest. This is usually only in principle rather than in practice, except in special tractable cases. We'll look at one example in detail in section 12.3, the hidden Markov model. The setup is that there is a hidden sequence $(Z_0, Z_1, \dots)$ which we'd like to infer, but at each timepoint n we only have a noisy readout X_n of the hidden value Z_n. An application is denoising GPS traces, to infer a true path.

* ✻ *

Random process modelling is (formally) just another instance of generative modelling. It can be approached in exactly the same way we've approached every single probability modelling problem throughout this course: first invent a parameterized distribution $\Pr_X(x\,;\,\theta)$ for a sequence-value random variable X, then fit the model to a dataset of sequences by tuning the unknown parameter θ. As with any generative modelling task, we can evaluate how good our model is by measuring the log likelihood of the dataset.

generative modelling, section 1.6 page 22

What makes random process modelling distinctive is that the models we use involve generating the items in X one by one, rather than generating the whole of X in one go. It's this iterative generation that makes them useful for modelling systems that evolve, and for fitting 'unstructured' data such as variable-length text.

12.1. Learning a Markov chain

The simplest sequence model is a Markov chain, and 'fitting the model' just means estimating the transition probabilities.

It's easy to estimate the transition matrix from a discrete sequence $(x_0, x_1, \ldots, x_n)$: we just count up the number of transitions of each type. For example, from the sequence $aabbabaabbab$ we get the transition matrix

$$P = \begin{array}{c} \\ a \\ b \end{array} \begin{array}{c} \begin{array}{cc} a & b \end{array} \\ \begin{pmatrix} 2/6 & 4/6 \\ 3/5 & 2/5 \end{pmatrix} \end{array}.$$

This obvious idea is fine for Markov models with a finite state space. But how might we fit a Markov model with an uncountable state space, for example if we want a model for a sequence of floating point numbers?

One of the big ideas in this course is that all models can be fitted in exactly the same way: by maximum likelihood estimation. We simply write out the likelihood of the observed dataset, and then choose unknown parameters to maximize the likelihood. This procedure works for Markov chains (and in fact the obvious idea of counting up transitions is a special case). All we need for it is a formula for the likelihood. It is derived as follows:

$$\begin{aligned}
\Pr(x_0, x_1, \ldots, x_n) &= \Pr(x_n \mid x_0, \ldots, x_{n-1}) \Pr(x_0, \ldots, x_{n-1}) \quad \text{by defn of conditional probability} \\
&= \Pr(x_n \mid x_{n-1}) \Pr(x_0, \ldots, x_{n-1}) \quad \text{by memorylessness} \\
&= \cdots \\
&= \Pr(x_0) \prod_{i=1}^{n} \Pr(x_i \mid x_{i-1}) \quad \text{by induction.}
\end{aligned}$$

When we do maximum likelihood estimation, the algebra ends up looking like regression of x against itself (specifically, a regression of x_i against x_{i-1}), so this type of fit is called 'autoregressive' modelling. Another name is 'teacher forcing'.

Example 12.1.1 (Autoregressive fit).
Let $(x_0, x_1, \ldots, x_n)$ be a time series which we believe is generated by

$$X_{i+1} = a + bX_i + \text{Normal}(0, \sigma^2), \qquad 0 \le i < n.$$

Estimate a, b, and σ.

The log likelihood of the entire dataset is

$$\log \Pr(x_0, x_1, \ldots, x_n) = \log \Pr_{X_0}(x_0) + \sum_{i=1}^{n} \log \Pr_{X_i}(x_i \mid x_{i-1}).$$

The question doesn't tell us anything about the distribution of X_0, and it certainly doesn't tell us that that distribution depends on a or b or σ. So it's reasonable to interpret the question as implying that $\Pr_{X_0}$ doesn't depend on those parameters. Hence, for the purposes of estimating those parameters, all we need to maximize is the second term. Tweaking the notation to emphasize the parameters we're maximizing over, this term is

$$\begin{aligned}
&\sum_{i=1}^{n} \log \Pr_{X_i}(x_i \mid X_{i-1} = x_{i-1}\,;\, a, b, \sigma) \\
&\qquad = \sum_{i=1}^{n} \log \Pr_{N(a+bx_{i-1}, \sigma^2)}(x_i) \quad \textit{since } X_i \sim N(a + bx_{i-1}, \sigma^2) \\
&\qquad = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^{n} \big(x_i - (a + bx_{i-1})\big)^2.
\end{aligned}$$

This is nothing other than a linear regression, predicting the responses x_i as a function of the predictor variables x_{i-1}, which we've solved many times already.

□

12.2. Recurrent neural networks

The task we considered at the beginning of section 12 was to learn a model for baby names, from a dataset

aaron	abraham	aditya
abdullah	adam	adonis
abel	addison	adrian
abigail	adeline	...

Let's write a string x as a sequence of characters, $x = x_1 x_2 \cdots x_n$. We want to be able to generate random sequences $X = X_1 X_2 \cdots X_N$ where both the characters and the length are random.

Recurrent neural networks are a modern model for such sequences. They derive from a language model proposed by Andrei Markov, the inventor of Markov chains. Let's first review his model.

MARKOV MODELS

> Consider a Markov chain with state space consisting of the alphabet a–z plus two extra characters, $\varnothing$ to denote 'start of string' and $\square$ to denote 'end of string'. Generate random strings by starting at $\varnothing$ and jumping from character to character until we hit $\square$. We can depict this as
>
> $$\varnothing \to X_1 \to X_2 \to \cdots \to X_N \to \square$$
>
> and the likelihood function is
>
> $$\Pr(x_1 x_2 \cdots x_n) = \Pr(x_1 \mid \varnothing)\ \Pr(x_2 \mid x_1) \times \cdots \times \Pr(\square \mid x_n).$$
>
> The character-to-character transition probabilities $\Pr(x_i \mid x_{i-1})$ might be estimated by simply counting transitions in the dataset, or they might be fitted from a parametric model $\Pr(x_i \mid x_{i-1}\ ;\ \theta)$, as in the autoregressive model from section 12.1.

The trouble with the simple character-by-character Markov model is that it's too crude: it'll produce names that don't make sense. For example, it needs to allow the transition a→a to accommodate names like Aaron; but it thereby allows the substring 'aaa', which never actually occurs in the dataset and which we'd prefer not to allow. We'll get better results if we include some history.

> Andrei Markov suggested a trigram-based model for fingerprinting the characteristic writing style of an author: he proposed generating the next character randomly based on the preceding *two* characters. This is able to express the rule "substring 'aa' is allowed but not the substring 'aaa'."
>
> $$\begin{tikzcd} \varnothing \arrow[rrrrrrr, bend left] & \varnothing \arrow[r] \arrow[rrrrrr, bend left] & X_1 \arrow[r] \arrow[rrrrr, bend left] & X_2 \arrow[r] \arrow[rrrr, bend left] & X_3 \arrow[r] \arrow[rrr, bend left] & X_4 \arrow[r] \arrow[rr, bend left] & \cdots \arrow[r] \arrow[r, bend left] & X_N \arrow[r] & \square \end{tikzcd}$$
>
> We start with two $\varnothing$ tokens, then generate X_1 randomly. Then, based on $(\varnothing X_1)$ we generate X_2 randomly. And so on. To learn the appropriate distribution for generating the next character we can simply count up all the trigrams in the text, i.e. all sequences of three characters.

The diagram for the trigram model is a real mess, hard to do calculations with. Markov chains are much more amenable to calculations. Markov pointed out that his trigram model is in fact a Markov chain:

> Consider a Markov chain whose state space consists of *pairs* of tokens. The rule for generating the next pair has two parts: from state $(X_{i-1}, X_i) = (x_{i-1}, x_i)$ we first generate X_{i+1} randomly, and then we form the new state (x_i, X_{i+1}). Repeat, until we generate $\square$. Markov's trigram model can equivalently be written as
>
> $$\varnothing\varnothing \to \varnothing X_1 \to X_1 X_2 \to \cdots \to X_{N-1} X_N \to X_N \square$$
>
> with the implied requirement that every transition $(v, w) \to (x, y)$ with $w \neq x$ shall have probability 0. Since this process is a Markov chain, we can write out an explicit formula for

the probability of generating a given string $x_1 x_2 \ldots x_n$:

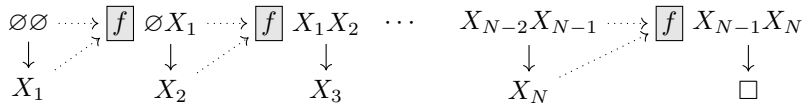
$$\Pr(x_1 x_2 \cdots x_n) = p(x_1 \mid \varnothing\varnothing)\, p(x_2 \mid \varnothing x_1) \times \cdots \times p(\square \mid x_{n-1} x_n).$$

Why just two characters, why not more history? And then, how much history should we keep? The more we keep, the more faithful our random sequences can be to the training dataset—but if we're too faithful then we'll just regurgitate the training data rather than generate novel sequences; and a random generator that can only regurgitate what it's seen will score badly when we evaluate the log likelihood of unseen strings in the holdout dataset.

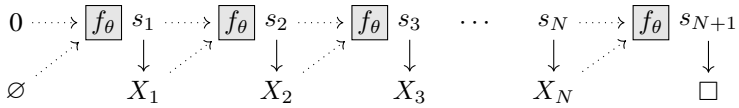
RECURRENT NEURAL NETWORKS TO DIGEST HISTORY

It's useful to be able to keep track of the process's history, but it's hard to know exactly how history should be represented. What if we could train a neural network to learn this? That's exactly the goal of the recurrent neural network probability model.

To see where the idea comes from, let's first redraw Markov's trigram model. In this depiction of his model, solid arrows indicate generating a random variable, and f is the simple deterministic book-keeping function that takes the previous state, and the newly generated token, and computes the next state.



What if, instead of a hard-coded history window, we have a more flexible 'history digest' s_i that can have as many dimensions as we like? We'll need to learn some flexible function to replace the simple deterministic book-keeping function:



Here, f_θ is some neural network that takes in the latest token X_{i-1} and the previous history digest s_{i-1}, and computes a new history digest $s_i = f_\theta(s_{i-1}, X_{i-1})$. The next event X_i is then generated randomly, using s_i as its parameters. To save the bother of having to specify separately how X_i is to be generated, we might as well split s_i into two parts, $s_i = (p_i, v_i)$ where p_i specifies the likelihood of X_i and v_i is used for computing s_{i+1}. This leads to the basic recurrence equations

$$(p_i, v_i) = f_\theta(x_{i-1}, v_{i-1}), \qquad X_i \sim \mathrm{Cat}(p_i).$$

Log likelihood and training. This model, like any other generative probability model, can be trained using maximum likelihood estimation, and it can be evaluated by computing the likelihood of a holdout dataset. What makes this easy is that there is an explicit formula for the log likelihood of a given string. The formula comes from conditional probability, along similar lines to the derivation for Markov chains:

$$\begin{aligned}\Pr(x_1 x_2 \cdots x_n) = \Pr_{X_1}(x_1) \times \Pr_{X_2}(x_2 \mid x_1) \times \Pr_{X_3}(x_3 \mid x_1 x_2) \times \\ \cdots \times \Pr_{X_n}(x_n \mid x_1 \cdots x_{n-1}) \times \Pr_{X_{n+1}}(\square \mid x_1 \cdots x_n).\end{aligned}$$

When we're generating the next character X_i, its distribution p_i is a deterministic function of $x_1 x_2 \cdots x_{i-1}$ thanks to the structure of the recurrent neural network, and therefore the probability $\mathbb{P}(X_i = x_i | x_1 \cdots x_{i-1})$ is easily computable. It's daunting to write it out in algebra, much cleaner in pseudocode:

```
def loglik(x):
    res = 0
    lastchar,v = ∅,0
    for char in x + "□":
        p,v = f_θ(lastchar, v)
        char_log_lik = log(p[char])
        res = res + char_log_lik
        lastchar = char
    return res
```

Generating random sequences. Once we've trained the neural network f_θ it's easy to generate a random string.

```
def generate():
    x,v = "∅",0
    while x[−1] ≠ □:
        p,v = f_θ(x[−1], v)
        newchar = np.random.choice(ALPHABET□, p)
        x = x + newchar
    return x[1:−1]  # strip out ∅ and □
```

RNN IN CODE

Here is PyTorch code, a bit more fussy than our pseudocode. The convention for PyTorch sequence operations is to act on tensors of shape $n \times b \times d$ where we're working on a batch of b sequences at a time, n is the maximum length of these sequences, and d is the dimensionality of each item in a sequence. Also, PyTorch manages the iteration for us: we just call $f_\theta(x_0 x_1 \cdots x_{n-1}, v_0)$ and it doesn't just return $(p_1, v_1) = f_\theta(x_0, v_0)$ it returns $((p_1, \ldots, p_n), v_n)$. The particular class of f_θ functions that we're using here is called GRU, which was designed so that it's easy to learn to propagate the v state forwards.

```
1  url = "https://www.cl.cam.ac.uk/teaching/2223/DataSci/data/english_names.txt"
2  names = pandas.read_table(url, header=None, names=['X']).X.str.lower()
3  alphabet = list(set(''.join(names)))
4  alphabet_n = {c:i+1 for i,c in enumerate(alphabet)}
5  A = len(alphabet)+1
6
7  # Convert from strings to integer-coding (with 0 at beginning and end) then one-hot coding
8  names_n = [[0] + [alphabet_n[x] for x in name] + [0] for name in names]
9  names_n = [torch.tensor(n) for n in names_n]
10 names_oh = [nn.functional.one_hot(n, num_classes=A).float() for n in names_n]
11
12 class RSeq(nn.Module):
13     def __init__(self, H=50, L=2):
14         super().__init__()
15         # Use rnn for the iterative part, producing a vector of dim. H per timestep,
16         # then apply an additional map before turning it into a probability distribution of dim. A
17         self.rnn = nn.GRU(input_size=A, hidden_size=H, num_layers=L, dropout=.05)
18         self.map = nn.Linear(self.rnn.hidden_size, A)
19
20     def f(self, x, v=None):  # x.shape m × b × A, v.shape L × b × H
21         assert x.shape[1]==1, "This code only works with batch_size=1"
22         y,v = self.rnn(x, v) # y.shape m × b × H, v.shape L × b × H
23         z = self.map(y)      # z.shape m × b × A
24         logp = nn.functional.log_softmax(z, dim=2) # logp.shape m × b × A
25         return logp,v
26
27     def forward(self, x): # x.shape = (n+1)*b*A, x assumed to have ∅ and □
28         logp,_ = self.f(x[:−1])      # logp.shape n × b × A
29         return torch.sum(logp * x[1:], dim=(0,2)) # shape b
30
31     def generate(self):
32         is_training = self.training
33         self.train(False)
34         with torch.no_grad():
35             res = ''
36             x_n,v = 0,None
37             while True:
38                 x_n = torch.tensor(x_n).reshape(1,1)
39                 x_oh = nn.functional.one_hot(x_n, num_classes=A).float()
40                 logp,v = model.f(x_oh, v)
41                 p = torch.exp(logp[0,0])
42                 x_n = np.random.choice(A, p=p.detach().numpy())
43                 if x_n == 0: break
```

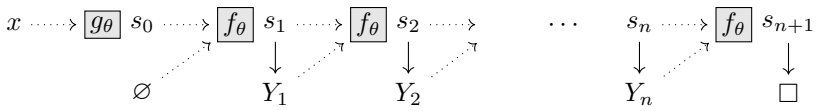
```
                res += alphabet[x_n-1]
        self.train(is_training)
        return res

model = RSeq()
model.train(True)
optimizer = optim.Adam(model.parameters())

with Interruptable() as check_interrupted:
    while True:
        for j in np.random.permutation(len(names_oh)):
            check_interrupted()
            optimizer.zero_grad()
            e = - model(names_oh[j][:,None,:])
            e.backward()
            optimizer.step()
```

* ✻ *

Recurrent neural networks easily be used for supervised learning tasks. Suppose for example we want to train a neural network to write captions for images. Let the the image be x, model the caption as a random sequence $Y = Y_1 Y_2 \cdots Y_n$, and fit a model with an extra neural network g_θ for processing the image and converting it to an initial state s_0.



The Transformer[75] architecture, a neural network that generates text much more impressively than does a recurrent neural network, also uses sequential generation with non-hidden state, but with an even simpler design. It generates each item in the sequence one by one, using the model $X_n \sim \mathrm{Cat}(f_\theta(x_1 x_2 \cdots x_{n-1}))$. The cleverness consists in designing a neural network that can accept an arbitrary length sequence as its input. But from the point of view of random variables, there's barely any difference between a recurrent neural network and a transformer.

[75] Ashish Vaswani et al. "Attention Is All You Need". In: *NIPS*. 2017. URL: http://arxiv.org/abs/1706.03762.

12.3. Hidden Markov models

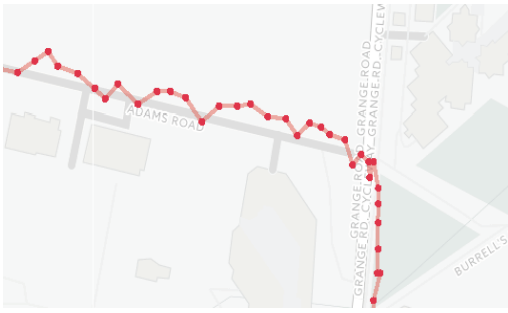
A *hidden Markov model*, also called a *latent Markov model*, is a two-layer model for a sequence of observations $X_0, X_1, \ldots$. We suppose there is some hidden sequence $Z_0, Z_1, \ldots$ which evolves according to a Markov chain, and assume that X_n is generated randomly based on the corresponding Z_n:

'latent' is from the Latin *latēre* to hide; compare to 'patent' from *patēre* to be open

$$\begin{tikzcd} Z_0 \arrow[r] \arrow[d] & Z_1 \arrow[r] \arrow[d] & Z_2 \arrow[r] \arrow[d] & Z_3 \arrow[r] \arrow[d] & \cdots \\ X_0 & X_1 & X_2 & X_3 & \end{tikzcd}$$

The thinking is that there are sequences $x_0 x_1 x_2 \cdots$ where it's very obvious that a simple memoryless model won't work—that deep history of earlier $x_0 x_1 \cdots x_{n-1}$ may be useful for predicting X_n—but where we have reason to believe there is some underlying physical process that *is* memoryless.

Example 12.3.1 (De-noising GPS).
Suppose we want to write an app to detect if the user is cycling, running, or driving, and which records or assists the user as appropriate. The GPS readings might look something like this (showing one sample per second).



A typical machine learning task is to estimate the user's true path[76] and mode of transport, given these readings. We might model this pair as a Markov chain, and model each GPS reading as a noisy function of the true location. A Markov model wouldn't work well for the GPS readings themselves: to infer whether egregious reading is noise or sudden movement, the recent past can help.

FITTING A HIDDEN MARKOV MODEL

The likelihood of an observed sequence $x_0 x_1 \cdots x_n$ is

$$\begin{aligned} \Pr(x_0 x_1 \cdots x_n) &= \sum_{z_0,\ldots,z_n} \Pr\big(x_0 x_1 \cdots x_n \mid z_0 z_1 \cdots z_n\big) \Pr\big(z_0 z_1 \cdots z_n\big) \\ &\qquad \text{by the law of total probability} \\ &= \sum_{z_0,\ldots,z_n} \Big(\prod_{i=0}^{n} q(x_i|z_i)\Big)\Big(p(z_0)\prod_{i=1}^{n} p(z_i|z_{i-1})\Big) \end{aligned}$$

where $p(z_i|z_{i-1})$ is the transition probability for the hidden state, $p(z_0)$ is the distribution of the initial hidden state, and $q(x_i|z_i)$ is the so-called *emission probability* describing the likelihood of observation x_i in hidden state z_i.

This likelihood function is horrid, completely intractable compared to the simple explicit formulae for Markov models and recurrent neural networks, on account of the sum over all possible hidden trajectories. It would be challenging to train a hidden Markov model (i.e. to learn p and q) from a dataset of x sequences.

Markov likelihood page 208, RNN likelihood page 210

[76] In a video from 2018, Uber engineer Karan Parikj explains how hidden Markov models are used to estimate a car's actual location given noisy GPS readings. `https://www.youtube.com/watch?v=ChtumoDfZXI`

BAYESIAN ANALYSIS

In general, latent variable models are a natural way to encode prior knowledge into a probability model. Hidden Markov models are natural when we have prior knowledge about the dynamics of the hidden Markov sequence and about the emission probabilities, and our goal is to infer the hidden state from observations, i.e. to find $(Z \mid X = x)$.

Most likely path. The Viterbi algorithm (which you learnt about in IA Machine Learning and Real World Data) computes the most likely hidden sequence given an observed sequence of readouts. In other words, it solves

$$\max_{z_0,\dots,z_n} \mathbb{P}\Big((Z_0,\dots,Z_n) = (z_0,\dots,z_n) \;\Big|\; (X_0,\dots,X_n) = (x_0,\dots,x_n)\Big).$$

It is for problems where Z has a finite state space. It assumes we know the transition probability matrix for Z, and we know $\Pr_{X_n}(x \mid Z_n = z)$, and we also know the distribution of the initial hidden state Z_0.

Bayesian analysis / particle filters. It can also be useful to compute the full distribution of hidden state, rather than just the most likely hidden state, using Bayes's rule. To make the problem concrete, suppose we want to find

$$\mathbb{P}\Big(Z_n = z_n \;\Big|\; (X_0,\dots,X_n) = (x_0,\dots,x_n)\Big)$$

where Z has a finite state space and we know its transition probability matrix

$$P_{zz'} = \mathbb{P}(Z_{n+1} = z' \mid Z_n = z)$$

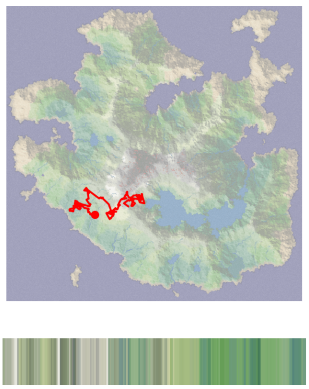
and we also know the distribution of the initial state $\Pr_{Z_0}(z)$ and the noise distribution $\Pr_{X_n}(x \mid Z_n = z)$. The example sheet asks you to solve this problem, in the context of the scampering stoat in example 12.3.2 below.

The example sheet also asks you to develop a computational solution, what's called a *particle filter*. The particle filter combines computational Bayesian methods with Markov chain tools. The idea is to have a weighted sample of values (a 'swarm of particles') drawn from the distribution of

$$(Z_i \mid X_0 = x_0, \dots, X_i = x_i)$$

and to update the swarm every timestep to incorporate the new reading x_{i+1}. Particle filters are popular in robotics, as a way for a robot to update its belief about its location given a stream of sensor readings.

Example 12.3.2.
Consider an object moving around in space, for example a stoat in the wild. Let Z_n be its location at timestep n. We might model this as a simple random walk. Or we might use a more elaborate model which includes velocity, or mode (slinking, gambolling, chasing, …).



Now, suppose we release the animal with a GPS tracker and a low-resolution camera—and then the GPS tracker breaks, but we still have a feed from the camera, and we know roughly what terrain X_n it was in each day (the bottom panel in the plot above). If we know the map that it's moving on, and we know the terrain it sees each day, can we estimate its current location?